

Software Development Guide

Enterprise Learning Management System (LMS)

Case Study Reference: MERN-004

Project Duration	4 Weeks
Team Size	2 Members
Methodology	Agile (1-Week Sprints)
Frontend	React (TailAdmin Template)
Backend	Node.js, Express.js, MongoDB
Document Type	Software Development Guide

Project Overview & Objectives

1.1 Background

Ezitech runs training programs, internships, certifications, bootcamps, workshops, and corporate learning across many technologies. As the number of students and courses grows, running everything on scattered tools has become hard to manage. This project builds a central Learning Management System (LMS) that will host courses, track student progress, manage assessments, issue certificates, and give instructors and admins clear reporting - all from one platform.

1.2 Project Objective

Build a working, production-style Enterprise LMS covering the core learning lifecycle: course delivery, learning paths, assessments, progress tracking, certificates, discussions, and analytics. The system should be built with clean, modular code so that it can later be extended into a full SaaS product.

1.3 Scope 4-Week Sprint (MVP)

The full case study describes a very large enterprise platform. With a 2-person team and a 4-week timeline, we cannot build every module to production depth. This guide defines a realistic MVP (Minimum Viable Product) that covers the most important modules well, instead of covering all modules poorly. Advanced items (bonus challenges, revenue/earnings, SCORM, offline downloads, multi-language) are marked as Phase 2 / stretch goals.

Priority	Modules Included in MVP (Weeks 1-4)
Must-Have	Authentication (JWT + RBAC), User Management (Student/Instructor/Admin), Course Management (categories, sections, lessons), Basic Learning Paths, Quizzes/MCQ Assessments, Student Progress Tracking, Student & Admin Dashboards
Should-Have	Certificate generation (PDF + QR verification), Discussion board (course-level Q&A), Notification Center (email + in-app), Basic reporting (PDF/CSV export)
Stretch / Phase 2	Coding assignment auto-evaluation, Live classes (WebRTC), Course marketplace, Coupons/subscriptions, SCORM support, Multi-language support, Instructor earnings

1.4 Success Criteria

- A student can register, log in, enroll in a course, complete lessons, attempt a quiz, and receive a certificate.
- An instructor can create a course, add sections/lessons, and grade assessments.
- An admin can view platform-wide analytics and manage users/courses.
- The codebase follows clean architecture with clear separation of concerns.
- APIs are documented (Swagger) and testable via a Postman collection.
- The application can be deployed using Docker.

2. Functional Modules

The table below summarizes the functional modules from the case study and how each is treated in this 4-week build.

Module	Key Features	Status in MVP
Authentication & Security	JWT auth, RBAC, session management, device login tracking	Included (MFA as stretch)
User Management	Students, Instructors, Mentors, Course Managers, Admins	Included (core 3 roles: Student, Instructor, Admin)
Course Management	Categories, course builder, sections, lessons, video/PDF content, assignments, prerequisites, tags	Included (core builder + content types)
Learning Path Management	Beginner/Intermediate/Advanced roadmaps, prerequisite validation	Simplified version included
Assessment System	Quizzes, MCQs, coding assignments, manual/auto evaluation, timed exams	Quizzes/MCQ included; coding auto-eval is stretch
Student Progress	Course completion, quiz performance, attendance, statistics	Included
Certificate Management	Dynamic templates, QR verification, certificate history	Included (basic template + QR)
Discussion & Community	Course discussion board, Q&A, announcements	Included (basic board)
Instructor Panel	Create courses, grade assignments, view analytics	Included
Student Dashboard	Active/completed courses, deadlines, certificates	Included
Admin Dashboard	Student growth, course performance, revenue, engagement analytics	Included (core analytics; revenue is stretch)
Notification Center	Enrollment, deadlines, quiz alerts, announcements	Included (email + Socket.IO in-app)
Reporting	Reports for students/courses/instructors, PDF/Excel/CSV export	Included (CSV/PDF export)

3. Tech Stack

3.1 Frontend

Technology	Purpose
React	Core UI library for building components
TailAdmin (React Template)	Pre-built admin dashboard UI (layouts, tables, charts, cards) - saves UI build time
TypeScript	Type safety across components, API responses, and state
Redux Toolkit	Global state management (auth, user, course, cart-like enrollment state)
React Query	Server-state management: data fetching, caching, background refresh
React Router	Client-side routing between dashboards, course pages, auth pages
Axios	HTTP client with interceptors for auth tokens and error handling

3.2 Backend

Technology	Purpose
Node.js	JavaScript runtime for the server
Express.js	REST API framework, routing, middleware
MongoDB	Primary NoSQL database
Mongoose	Schema modelling and validation for MongoDB
JWT (jsonwebtoken)	Access & refresh token based authentication
bcrypt	Password hashing
Redis	Caching, session/token blacklist, rate limiting
Socket.IO	Real-time notifications and discussion updates
Cloudinary	Storage for videos, PDFs, images (course content, certificates)
Nodemailer	Transactional emails (verification, deadlines, certificates)

3.3 Tooling & Quality

- ESLint + Prettier for consistent code style
- Postman for API testing and collection sharing
- Swagger (OpenAPI) for API documentation

- GitHub Actions (optional) for basic CI checks (lint + test on PR)
- Jest / Supertest for backend unit and integration tests

4. Agile Workflow & Sprint Plan (Week 1-4)

The project follows Agile with four 1-week sprints. Each sprint starts with a short planning session and ends with a review/demo and retrospective. Daily async check-ins (15 min or a chat message) keep both engineers aligned since the team is small.

4.1 Sprint Ceremonies (Lightweight, for a 2-person team)

Ceremony	Frequency	Duration	Purpose
Sprint Planning	Start of each week	30-45 min	Pick tasks from backlog, confirm ownership, define "Done"
Daily Check-in	Daily	10-15 min	Quick status: done yesterday, doing today, blockers
Mid-Sprint Sync	Mid-week	20 min	Catch integration issues early (API contracts, shared types)
Sprint Review / Demo	End of each week	30 min	Demo working features to supervisor
Retrospective	End of each week	15 min	What went well / what to improve next sprint

4.2 Sprint Plan Overview

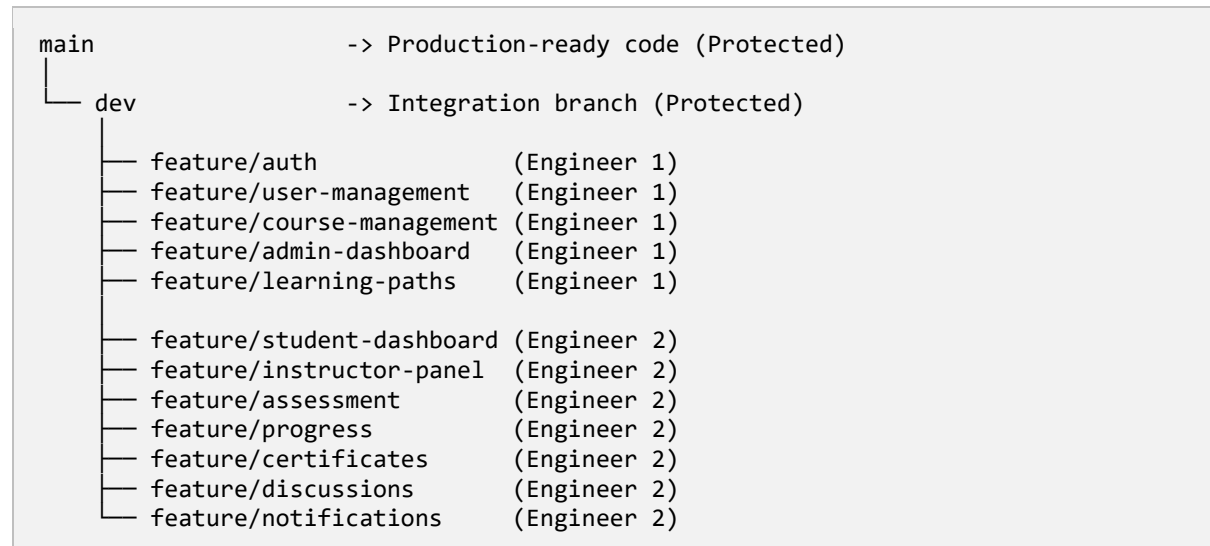
Sprint	Theme	Primary Goal
Week 1	Foundation & Setup	Repo, architecture, auth, DB schema, TailAdmin integration, CI skeleton
Week 2	Core Feature Build	Course management, learning paths, assessments, progress tracking (parallel by engineer)
Week 3	Feature Completion & Integration	Certificates, discussions, dashboards, notifications; connect frontend + backend end-to-end
Week 4	Testing, Deployment & Documentation	Bug fixes, testing, Docker deployment, docs, demo prep

Detailed day-by-day tasks per engineer are provided in Section 12.

5. Git & GitHub Workflow

A simple three-tier branching model keeps work isolated and main always deployable.

5.1 Branch Structure



5.2 Branch Naming Convention

Type	Pattern	Example
Feature	feature/<short-description>	feature/course-builder
Bug Fix	fix/<short-description>	fix/login-token-refresh
Hotfix (production)	hotfix/<short-description>	hotfix/cert-qr-crash
Chore / Config	chore/<short-description>	chore/eslint-setup

5.3 Pull Request (PR) Process

1. Create a feature branch from the latest dev.
2. Commit in small, meaningful steps using Conventional Commits (e.g. feat: add quiz submission API).
3. Push the branch and open a PR into dev (never directly into main).
4. PR description must include: what changed, how to test, and screenshots for UI changes.
5. The other engineer reviews the PR (at least 1 approval required before merge).
6. Resolve review comments, then squash-and-merge into dev.
7. At the end of each sprint, dev is merged into main once the build passes and the demo is confirmed stable.

5.4 Commit Message Convention

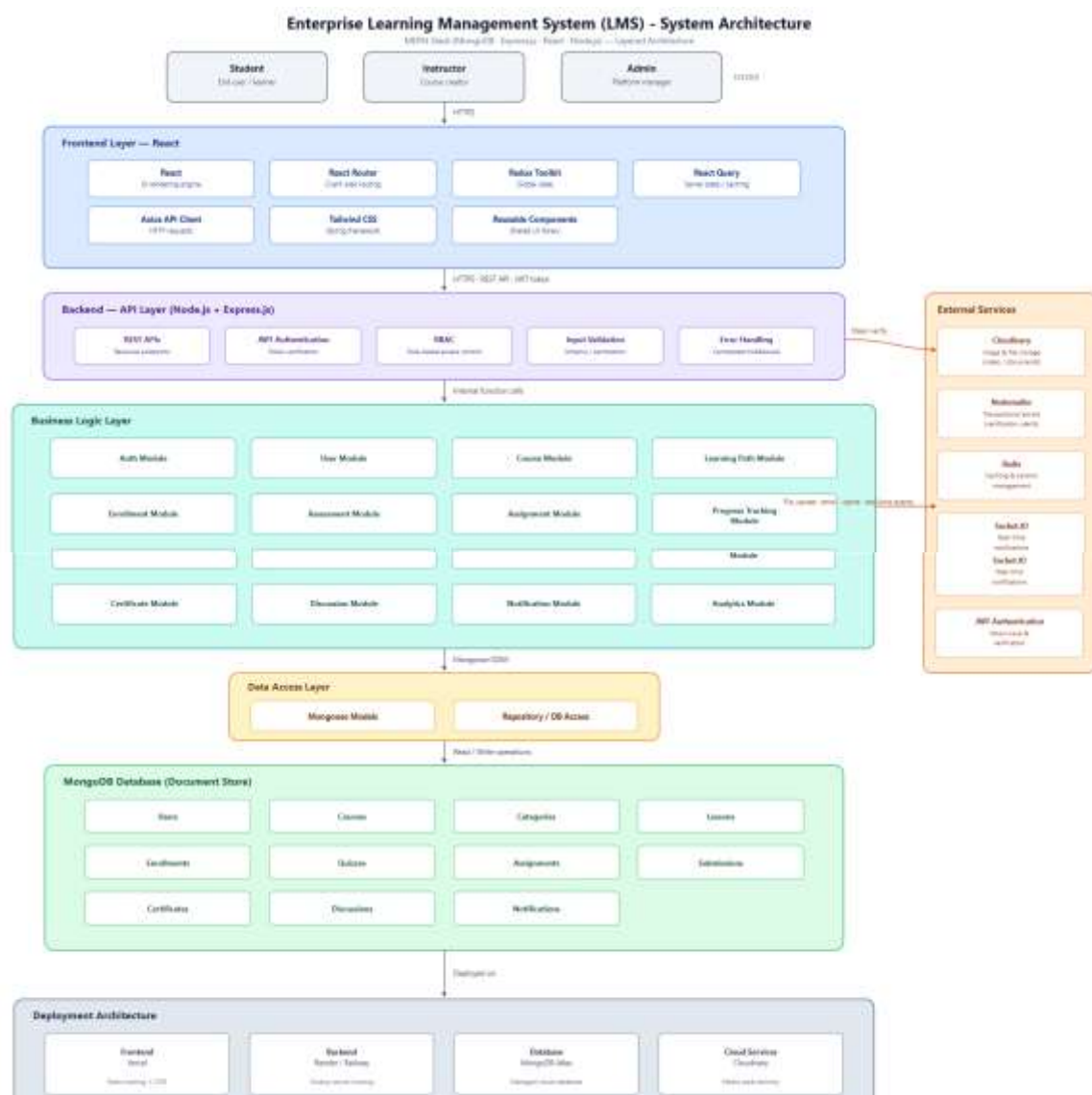
feat:	new feature	e.g. feat: add course enrollment API
fix:	bug fix	e.g. fix: correct quiz score calculation
refactor:	code change, no behavior	e.g. refactor: extract auth middleware

6.2 Frontend Structure

```
client/
├── src/
│   ├── components/      # SHARED: Card, Table, Button, Modal, Badge, Sidebar
│   ├── features/
│   │   ├── auth/        # Engineer 1
│   │   ├── users/       # Engineer 1
│   │   ├── courses/     # Engineer 1
│   │   ├── learning-paths/ # Engineer 1
│   │   ├── admin-dashboard/ # Engineer 1
│   │   ├── assessments/  # Engineer 2
│   │   ├── student-progress/ # Engineer 2
│   │   ├── certificates/ # Engineer 2
│   │   ├── discussions/  # Engineer 2
│   │   ├── notifications/ # Engineer 2
│   │   ├── student-dashboard/ # Engineer 2
│   │   └── instructor-dashboard/ # Engineer 2
│   ├── pages/           # route-level pages, one per dashboard/module
│   ├── app/             # SHARED: store.ts (Redux Toolkit), queryClient.ts
│   └── (React Query)
│       ├── routes/      # SHARED: AppRoutes.tsx, ProtectedRoute.tsx
│       ├── styles/      # SHARED: theme.ts, TailAdmin design tokens
│       └── main.tsx
├── .env.example
└── package.json
```


7. System Architecture & Request Flow

7.1 High-Level Architecture



7.2 Sample Request Flow - Student Submits a Quiz

1. Student clicks Submit on Quiz UI (React)
 - > React Query mutation calls POST /api/v1/assessments/:quizId/submit
2. Express receives request
 - > authMiddleware verifies JWT access token
 - > roleGuard confirms role = student
 - > validation middleware checks answer payload shape
3. assessments.controller -> assessments.service
 - > loads quiz + correct answers from MongoDB
 - > auto-evaluates MCQ answers, calculates score
 - > saves QuizAttempt document
 - > updates StudentProgress document
4. Side effects
 - > Socket.IO emits 'progress:updated' to the student's dashboard
 - > If quiz was the final requirement, a 'certificate:eligible' event fires
5. Response
 - > Standardized JSON response sent back to client
 - > React Query cache updated -> UI shows score instantly

8. Database Collections

MongoDB collections for the MVP scope, with key fields. Full field-level schemas (Mongoose) should be added to the technical appendix as development progresses.

Collection	Key Fields	Notes
users	name, email, passwordHash, role (student/instructor/admin), status, devices[]	Single collection with role field, or discriminators per role
courses	title, slug, category, description, instructorId, sections[], prerequisites[], tags[], status	Sections/lessons can be embedded or a separate collection
sections	courseId, title, order, lessons[]	Embedded in course if course size is small; separate if large
lessons	sectionId, title, type (video/pdf/assignment), contentUrl, order, duration	contentUrl points to Cloudinary asset
learningPaths	title, level (beginner/intermediate/advanced), courseSequence[], prerequisites[]	Simplified: an ordered list of course IDs
enrollments	studentId, courseId, enrolledAt, status, progressPercent	Bridges students and courses
quizzes	courseId/lessonId, questions[], timeLimitMinutes, passingScore	questions[] embeds MCQ options + correct answer

Collection	Key Fields	Notes
quizAttempts	quizId, studentId, answers[], score, submittedAt	One document per attempt
studentProgress	studentId, courseId, completedLessons[], quizScores[], attendance, lastActivityAt	Aggregated progress snapshot per enrollment
certificates	studentId, courseId, issuedAt, certificateUrl, qrCode, verificationCode	verificationCode used by the public verification portal
discussions	courseId, authorId, title, body, replies[]	replies[] can be embedded for MVP simplicity
notifications	userId, type, message, read, createdAt	Populated by backend events, pushed via Socket.IO

8.1 Relationship Notes

- One instructor can own many courses (1:N).
- One course has many sections, each with many lessons (1:N:N) - embed for MVP, normalize later if content grows large.
- Enrollment is the join between students and courses (N:M via a bridge collection).
- Each quiz attempt belongs to one student and one quiz; studentProgress aggregates results per course.
- Certificates reference both studentId and courseId, and are only created once progress reaches 100%.

9. API Standards

9.1 Versioning & Base URL

All routes are prefixed with a version so future breaking changes don't affect existing clients:

```
Base URL: /api/v1/
Example: /api/v1/courses/:courseId/enroll
```

9.2 REST Conventions

Method	Usage	Example
GET	Retrieve a resource or list	GET /api/v1/courses
POST	Create a new resource	POST /api/v1/courses
PUT	Full update of a resource	PUT /api/v1/courses/:id
PATCH	Partial update	PATCH /api/v1/courses/:id/status

Method	Usage	Example
DELETE	Remove a resource	DELETE /api/v1/courses/:id

9.3 Standard Response Format

Every response follows the same envelope so the frontend can handle it generically:

```
// Success
{
  "success": true,
  "message": "Course created successfully",
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 45 }
}

// Error
{
  "success": false,
  "message": "Validation failed",
  "errors": [ { "field": "email", "issue": "Invalid email format" } ]
}
```

9.4 HTTP Status Codes

Code	Meaning	When to Use
200	OK	Successful GET/PUT/PATCH/DELETE
201	Created	Successful POST that creates a resource
400	Bad Request	Validation errors
401	Unauthorized	Missing/invalid/expired token
403	Forbidden	Valid token but insufficient role/permission
404	Not Found	Resource does not exist
409	Conflict	Duplicate resource (e.g. email already registered)
500	Server Error	Unhandled/unexpected errors

9.5 Authentication Header

```
Authorization: Bearer <access_token>
```

Access tokens are short-lived (15 min); refresh tokens are stored as httpOnly cookies and used to silently renew access tokens via POST /api/v1/auth/refresh.

10. Shared UI/Theme & Coding Standards

10.1 UI/Theme Rules (TailAdmin)

- Keep TailAdmin's existing design tokens (colors, spacing, typography) - do not introduce a second design system.
- Centralize theme values (primary color, spacing scale) in one config so both engineers style consistently.
- Reuse TailAdmin's existing table, card, badge, and modal components before building new ones.
- All new components go in components/common/ if generic, or features/<feature>/components/ if feature-specific.
- Keep dashboards responsive - TailAdmin is mobile-friendly by default, so avoid fixed-pixel layouts that break it.

10.2 Coding Standards

Area	Standard
Naming	camelCase for variables/functions, PascalCase for components/classes, kebab-case for file/folder names (except React components)
TypeScript	No implicit any; define interfaces/types for all API request/response shapes in types/
Components	Function components only, one component per file, props typed with an interface
State	Redux Toolkit slices for global/shared state; local component state (useState) for UI-only concerns; React Query for all server data
Backend Structure	Controller -> Service -> Model layering; controllers stay thin (no business logic)
Error Handling	Centralized errorHandler middleware; services throw typed AppError objects, never raw strings
Validation	Joi or Zod schemas per route, validated in middleware before hitting the controller
Comments	Explain "why", not "what"; avoid redundant comments on self-explanatory code
Linting	ESLint + Prettier run on save and as a pre-commit hook (husky + lint-staged)

10.3 Environment & Secrets

- Never commit .env files - only .env.example with placeholder keys.
- All secrets (JWT secret, Cloudinary keys, SMTP credentials, DB URI) are read via process.env.
- Separate config values for development, staging, and production.

11. Team Responsibilities - Vertical Feature Ownership

Area	Engineer 1 - Core Platform & Content	Engineer 2 - Assessment, Progress & Engagement
Owns	Authentication & Security, User Management, Course Management, Learning Paths	Assessment System, Student Progress, Certificate Management, Discussion & Community, Notifications
Dashboards	Admin Dashboard	Student Dashboard & Instructor Dashboard
Shared Infra	Project setup, DB connection setup, Environment variables	Co-owns: TailAdmin setup, Shared UI components, Socket.IO, Redis, Testing & Deployment
Frontend Routes	/auth/*, /users/*, /courses/*, /learning-paths/*, /admin/*	/student/*, /instructor/*, /assessments/*, /progress/*, /certificates/*, /discussions/*, /notifications/*
Backend Modules	modules/auth, modules/users, modules/courses, modules/learning-paths, modules/admin, modules/reports	modules/assessments, modules/progress, modules/certificates, modules/discussions, modules/notifications, modules/instructor

11.1 Integration Points to Coordinate

- Engineer 1 exposes course/lesson IDs that Engineer 2's assessments and progress models depend on - agree on this schema in Week 1.
- Engineer 2's progress-completion event triggers certificate issuance, which Engineer 1's course completion data feeds into.
- Both engineers share the same auth middleware and role-guard utilities - built once by Engineer 1 in Week 1, consumed by both.

12. Detailed Weekly Tasks for Each Engineer

Each week is broken down by engineer. Tasks are sized to be realistic for a 5-day work week alongside daily check-ins.

Week 1 - Foundation & Setup

Day	Engineer 1 (Core Platform & Content)	Engineer 2 (Assessment, Progress & Engagement)
Day 1	Set up monorepo/repo structure, dev/main branches, ESLint/Prettier, install TailAdmin template	Install & configure TailAdmin template, set up base layout shell and shared UI component library
Day 2	Design DB schema (users, courses) and share ER diagram with Engineer 2 for review	Set up Redis connection, Socket.IO server skeleton, base notification model
Day 3	Build JWT auth: register/login/refresh, password hashing, RBAC middleware	Build error-handling middleware, standardized API response utility, request validation setup
Day 4	Build role guard + protected routes; integrate TailAdmin layout shell (sidebar/header) in frontend	Integrate TailAdmin layout shell (sidebar/header) with auth-protected routing; write Postman collection skeleton + Swagger base config;
Day 5	Sprint review: demo login/register flow end-to-end; retrospective	Sprint review: demo TailAdmin shell + notification/socket test event; retrospective

Week 2 - Core Feature Build

Day	Engineer 1 (Core Platform & Content)	Engineer 2 (Assessment, Progress & Engagement)
Day 1	Build Course model + CRUD API (create/read/update/delete courses)	Build Quiz model + CRUD API (create quiz, add MCQ questions)
Day 2	Build Section & Lesson models, nested CRUD, Cloudinary upload for video/PDF	Build quiz submission API + auto-evaluation logic for MCQs
Day 3	Build course builder UI (React) - create course, add sections/lessons	Build StudentProgress model + update-on-activity logic
Day 4	Build Learning Path model + simple roadmap UI (ordered course list)	Build quiz-taking UI (timer, question navigation, submit)
Day 5	Sprint review: demo course creation + content upload; retrospective	Sprint review: demo quiz attempt + auto-scoring; retrospective

Week 3 - Feature Completion & Integration

Day	Engineer 1 (Core Platform & Content)	Engineer 2 (Assessment, Progress & Engagement)
Day 1	Build enrollment flow (student enrolls in course, prerequisite check)	Build Certificate model + PDF generation + QR code embedding
Day 2	Build Admin Dashboard backend (user/course management APIs, analytics aggregation)	Build certificate verification portal (public route + verification code lookup)

Day	Engineer 1 (Core Platform & Content)	Engineer 2 (Assessment, Progress & Engagement)
Day 3	Build Admin Dashboard UI (analytics views, user/course management screens)	Build Instructor Dashboard (course list, grading assignments view)
Day 4	Build Reporting module (CSV/PDF export APIs) for admin reports	Build Notification Center UI + wire Socket.IO events end-to-end
Day 5	Integration day: connect frontend/backend across both verticals, fix contract mismatches	Integration day: joint testing of full student journey (enroll -> learn -> quiz -> certificate)

Week 4 - Testing, Deployment & Documentation

Day	Engineer 1 (Core Platform & Content)	Engineer 2 (Assessment, Progress & Engagement)
Day 1	Write unit/integration tests for auth, courses, learning paths	Write unit/integration tests for assessments, progress, certificates
Day 2	Fix bugs from test results; polish course builder UI/UX	Fix bugs from test results; polish quiz UI and certificate template
Day 3	Finalize Docker Compose (frontend + backend + MongoDB + Redis); deploy to staging	Finalize Swagger docs + Postman collection; add admin analytics reports (CSV/PDF export)
Day 4	Write README, deployment guide, and system architecture doc	Write user manual (student/instructor/admin walkthroughs) and API documentation
Day 5	Final rehearsal of live demo; prepare technical presentation slides	Final rehearsal of live demo; prepare technical presentation slides

13. Testing, Deployment & Integration Strategy

13.1 Testing Strategy

Test Type	Tooling	What It Covers
Unit Tests	Jest	Services and utility functions in isolation (e.g. quiz scoring logic, token generation)
Integration Tests	Jest + Supertest	API endpoints against a test MongoDB instance (in-memory or Docker test container)
Manual/API Testing	Postman Collection	Full request/response verification per module, shared across the team
Frontend Component Tests	React Testing Library (optional, if time allows)	Key components: login form, quiz UI, dashboard cards

Test Type	Tooling	What It Covers
Manual E2E Walkthrough	Human testing (both engineers)	Full student journey and instructor/admin journeys before each demo

13.2 Integration Strategy

- Agree on API contracts (request/response shapes) before building UI screens - avoids rework.
- Use a shared Postman environment so both engineers test against the same base URL and tokens.
- Merge feature branches into dev frequently (at least every 1-2 days) to avoid large, risky merges.
- Run the full app locally via Docker Compose before every sprint review to catch integration issues early.

14. Documentation Checklist & Final Deliverables

This checklist mirrors the deliverables required by the case study. Use it as a submission gate before the final demo.

#	Deliverable	Owner	Status
1	Complete Source Code (frontend + backend repos)	Both	[]
2	API Documentation (Swagger)	Engineer 2	[]
3	Database Schema (Mongoose models + description)	Engineer 1	[]
4	ER Diagram	Engineer 1	[]
5	System Architecture Diagram	Engineer 1	[]
6	Postman Collection	Engineer 2	[]
7	README (setup instructions, scripts, env variables)	Both	[]
8	Deployment Guide (Docker instructions, environment setup)	Engineer 1	[]
9	User Manual (student/instructor/admin walkthroughs)	Engineer 2	[]
10	Technical Presentation (slides)	Both	[]
11	Live Demonstration (rehearsed run-through)	Both	[]

14.1 Evaluation Criteria Reference

For context, submissions are evaluated against these weighted criteria from the case study - keep this in mind throughout development, not just before submission:

Criteria	Weight
Software Architecture	20%
Business Logic	20%
Database Design	15%
Security	15%
Performance	10%
UI/UX	10%
Documentation	5%
Presentation	5%

14.2 Final Pre-Submission Checklist

- All environment variables documented in .env.example (no secrets committed).
- Both main and dev branches are up to date and buildable without errors.
- Docker Compose runs the full stack from a clean clone.
- Swagger docs and Postman collection match the actual implemented endpoints.
- Demo data seeded for a smooth live demonstration.
- Known issues / out-of-scope items listed clearly in the README (honesty here is valued over silence).